ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH (CO5085)

Báo cáo Bài tập lớn A1

CNN và Transformer trên tập ảnh lớn

GVHD:	TS. Lê Thành Sách	
	TS. Nguyễn Quốc Minh	
Sinh viên:	Lê Hữu Trọng	2470578
	Huỳnh Văn Tâm Thiện	2470094

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 11/2026

Mục lục

1 Giới thiệu **2**

2 Tập dữ liệu **2**

2.1 Lựa chọn tập dữ liệu và lý do 2

2.2 Giấy phép và nguồn 3

2.3 Thống kê tập dữ liệu 3

2.4 Phân chia tập dữ liệu 3

2.5 Phân tích khám phá dữ liệu (EDA) 4

3 Chuẩn bị dữ liệu huấn luyện **5**

3.1 Tiền xử lý 5

3.2 Tăng cường dữ liệu 5

3.3 Cấu hình DataLoader 7

4 Cài đặt thí nghiệm **7**

4.1 Các mô hình 7

4.1.1 ResNet-50 7

4.1.2 DeiT-S 9

4.2 Các chế độ đóng băng 10

4.3 Siêu tham số huấn luyện 11

4.4 Độ đo đánh giá 11

5 Kết quả và phân tích **12**

5.1 Độ chính xác 12

5.2 Quá trình huấn luyện và chi phí 14

5.3 Phân tích lỗi 15

5.4 Giải thích dự đoán của mô hình 18

6 Thảo luận, kết luận và hạn chế **21**

6.1 Kết luận chính 21

6.2 Hạn chế 21

6.3 Khả năng tái lập 22

Tài liệu **22**

1 Giới thiệu

Bài tập lớn A1 so sánh một mạng tích chập (CNN) và một Vision Transformer (ViT) trên một tập ảnh đủ lớn và đủ khó để khác biệt giữa hai họ kiến trúc, và giữa các cách dùng trọng số pretrained, có ý nghĩa. Bài chọn **Food-101** [1] (101 món ăn, 101 000 ảnh), **ResNet-50** [3] cho họ CNN và **DeiT-S** [4] cho họ Transformer. Mỗi mô hình được huấn luyện theo bốn chế độ: từ đầu (*scratch*), và ba cách dùng trọng số pretrained ImageNet-1k là chỉ train classification head (*head*), đóng băng nửa đầu mạng (*partial*) và finetune toàn bộ (*full*).

- **Tập dữ liệu.** Food-101, 101 lớp món ăn. Sau khi tách train, validation và test có 68 175, 7 575 và 25 250 ảnh (Mục 2).
- **Mô hình.** ResNet-50 (`microsoft/resnet-50`) cho CNN và DeiT-S (`facebook/deit-small-patch16-224`) cho Transformer, cả hai là bản công bố trên Hugging Face, có trọng số ImageNet-1k. Chế độ scratch không nạp trọng số đó (Mục 4.1).
- **Chế độ huấn luyện.** Train từ đầu (*scratch*), chỉ train classification head (*head*), đóng băng nửa đầu mạng (*partial*) và finetune toàn bộ (*full*). Hai mô hình nhân bốn chế độ thành 8 run, seed 42 (Mục 4.2).
- **Tiền xử lý.** Một pipeline tăng cường và chuẩn hoá, dùng chung cho mọi mô hình và mọi chế độ (Mục 3).

2 Tập dữ liệu

2.1 Lựa chọn tập dữ liệu và lý do

Food-101 [1] gồm 101 lớp món ăn, mỗi lớp 1 000 ảnh (750 ảnh train và 250 ảnh test theo split chính thức), cạnh dài nhất của ảnh không quá 512 px. Ảnh test đã được kiểm tra thủ công; ảnh train cố ý để nguyên nên còn nhiễu, chủ yếu là màu quá gắt và đôi khi sai nhãn. Food-101 được chọn vì bốn lý do:

- **Đủ lớn và đủ khó.** 101 lớp, 75 750 ảnh train chính thức; bài toán phân loại chi tiết, nhiều món khác nhau trông gần giống nhau.
- **Không trùng với ImageNet.** Tiny-ImageNet, Imagenette và các tập con của ImageNet bị loại vì ảnh của chúng nằm trong dữ liệu mà mô hình pretrained đã học, làm lệch so sánh scratch với pretrained.
- **Hợp với mô hình pretrained.** Ảnh tự nhiên, dùng được ở độ phân giải 224 px; CIFAR-100 (32×32) phải phóng to gấp 7 lần nên bị loại.
- **Phù hợp ngân sách.** Khoảng 68 000 ảnh train đủ nhỏ để train từ đầu trên một GPU A10G.

Lưu ý: ImageNet-1k có sẵn một số lớp món ăn trùng hoặc gần với Food-101, nên trọng số pretrained có lợi thế ngữ nghĩa ở các lớp này.

2.2 Giấy phép và nguồn

Food-101 không có giấy phép mở: trên thẻ dữ liệu Hugging Face do nhóm tác giả ở ETH Zürich đăng, mục giấy phép ghi là không xác định (*unknown*). Ảnh được thu thập từ trang Foodspotting và thuộc về người đã đăng; theo mục *License Agreement* của thẻ dữ liệu, mọi cách dùng vượt quá phạm vi sử dụng hợp lý cho nghiên cứu khoa học (*scientific fair use*) phải thoả thuận với chủ ảnh. Bài tập chỉ dùng dữ liệu để học tập và nghiên cứu, không phân phối lại ảnh. Nguồn và điều khoản của Food-101 được ghi ở Bảng 1; dữ liệu được tải về qua thư viện torchvision.

Hạng mục	Liên kết	Nội dung được dùng
Food-101	huggingface.co/datasets/ethz/food101	Thẻ mô tả do nhóm ETH Zürich đăng: 101 lớp, 750 ảnh train và 250 ảnh test mỗi lớp
Điều khoản sử dụng	huggingface.co/datasets/ethz/food101#license-agreement	Không có giấy phép mở. Dữ liệu chỉ được dùng để học và nghiên cứu

Bảng 1: Nguồn và điều khoản sử dụng của Food-101.

2.3 Thống kê tập dữ liệu

Bảng 2 tóm tắt quy mô của tập dữ liệu; thống kê chi tiết hơn nằm ở Mục 2.5.

Thuộc tính	Giá trị
Số lớp	101
Tổng số ảnh	101 000 (75 750 train, 25 250 test)
Độ phân giải	cạnh dài 512 px, cạnh ngắn 122–512 px
Chế độ màu	100 997 ảnh RGB, 3 ảnh xám

Bảng 2: Thống kê tập dữ liệu.

2.4 Phân chia tập dữ liệu

Split test chính thức được giữ nguyên. Validation được tách từ split train chính thức: 10% mỗi lớp, phân tầng theo nhãn, seed 42 (Bảng 3). Split này cố định cho mọi run, ở cả hai mô hình và cả bốn chế độ, nên scratch và pretrained được so sánh trên cùng một split.

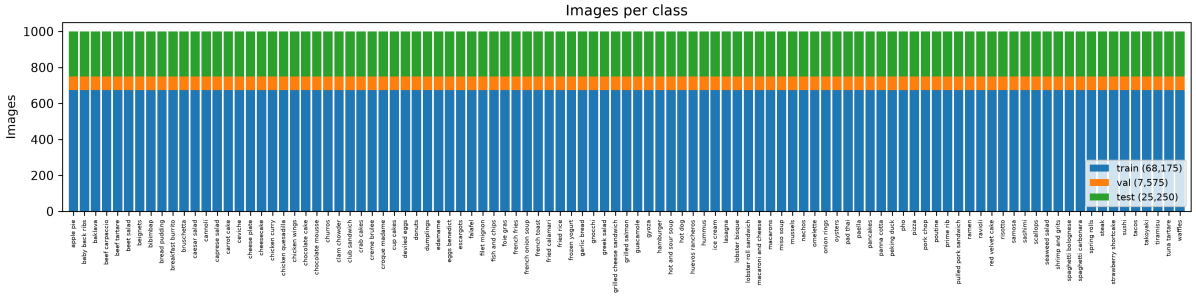
Tập	Số ảnh	Ảnh/lớp	Nguồn
train	68 175	675	90% split train chính thức
validation	7 575	75	10% split train chính thức
test	25 250	250	split test chính thức

Bảng 3: Ba tập dùng trong A1.

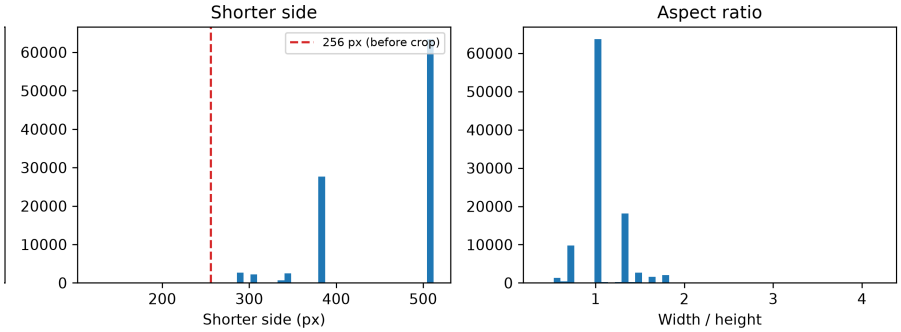
- **Test không tham gia quyết định nào:** epoch giữ lại và learning rate đều chọn trên validation; test chỉ được chấm sau khi train xong, trên checkpoint đã cố định.
- **Validation lấy từ train, không từ test,** để test không tham gia chọn mô hình.
- **Nhiễu nhãn:** validation lấy từ train nên cũng nhiễu, còn test đã được kiểm tra thủ công; validation accuracy vì vậy có thể thấp hơn test accuracy.

2.5 Phân tích khám phá dữ liệu (EDA)

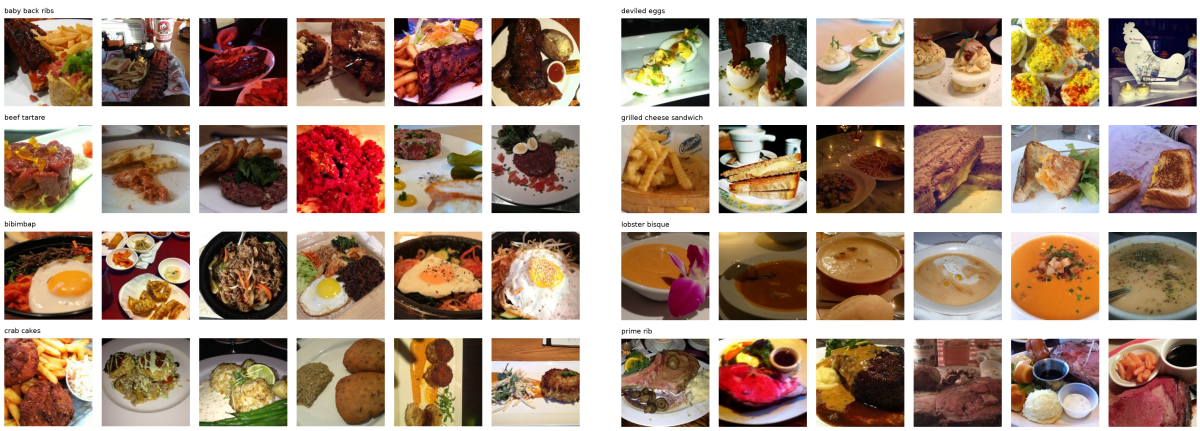
Kích thước ảnh được đo trên toàn bộ 101 000 ảnh gốc, trước khi thu nhỏ; mean và std theo kênh được tính trên toàn bộ 68 175 ảnh train, sau bước cắt cố định (Hình 1–3).



Hình 1: Số ảnh mỗi lớp trong train, validation và test.



Hình 2: Kích thước ảnh gốc. Trái: số ảnh theo độ dài cạnh ngắn (vạch đỏ là 256 px, mức đổi cạnh ngắn trước khi cắt). Phải: số ảnh theo tỉ lệ khung (rộng/cao).



Hình 3: Ảnh train của một số lớp, sau bước cắt cố định.

Nhận xét:

- **Cân bằng lớp.** Mỗi lớp có đúng 675 ảnh train, 75 ảnh validation và 250 ảnh test (Hình 1), nên không cần trọng số lớp.
- **Kích thước.** Cạnh dài luôn là 512 px. Theo Hình 2, khoảng 63% ảnh là ảnh vuông 512×512 và khoảng 27% có cạnh ngắn 384 px (khung 4:3); các tỉ lệ này đọc từ biểu đồ nên chỉ xấp xỉ. Chỉ 44 ảnh (0.04%) có cạnh ngắn dưới 256 px, được phóng lên khi đổi cạnh ngắn về 256 px, và quá ít để ảnh hưởng kết quả. Phép cắt giữa bỏ phần rìa của ảnh, đáng kể nhất với ảnh có khung dài (tỉ lệ đến 4.2), nhưng các ảnh này rất hiếm.
- **Chế độ màu.** 100 997 ảnh RGB và 3 ảnh xám (chuyển sang RGB khi tiền xử lý).
- **Màu sắc.** Mean RGB là $(0.557, 0.442, 0.327)$, std khoảng 0.26: ảnh món ăn nghiêng về đỏ hơn ImageNet $(0.485, 0.456, 0.406)$.
- **Nhiễu nhãn** (Hình 3). Ảnh chụp trong nhà hàng nên ánh sáng, góc chụp và nền rất khác nhau. Trong 48 ảnh mẫu có ít nhất hai ảnh mà nhãn không ứng với chủ thể rõ ràng (ảnh thứ hai của *bibimbap* chủ yếu là há cảo; ảnh thứ ba của *grilled cheese sandwich* không thấy bánh) và vài ảnh có màu rất gắt, đúng như tác giả mô tả. Mẫu quá nhỏ để ước lượng tỉ lệ nhiễu.
- **Cặp lớp dễ nhầm**: các món gần nhau như *prime rib*, *steak*; sẽ được đo bằng ma trận nhầm lẫn (Mục 4.4).

3 Chuẩn bị dữ liệu huấn luyện

3.1 Tiền xử lý

Mọi mô hình và mọi chế độ dùng chung một pipeline dữ liệu, các run chỉ khác nhau ở mô hình và cách huấn luyện, không khác ở dữ liệu đưa vào.

Cắt cố định, áp dụng một lần cho mọi ảnh. Mọi ảnh của cả ba tập (train, validation, test) được xử lý đúng một lần theo cùng một cách: chuyển sang RGB, đổi cạnh ngắn về 256 px, rồi cắt phần giữa 224×224 và lưu lại dạng JPEG.

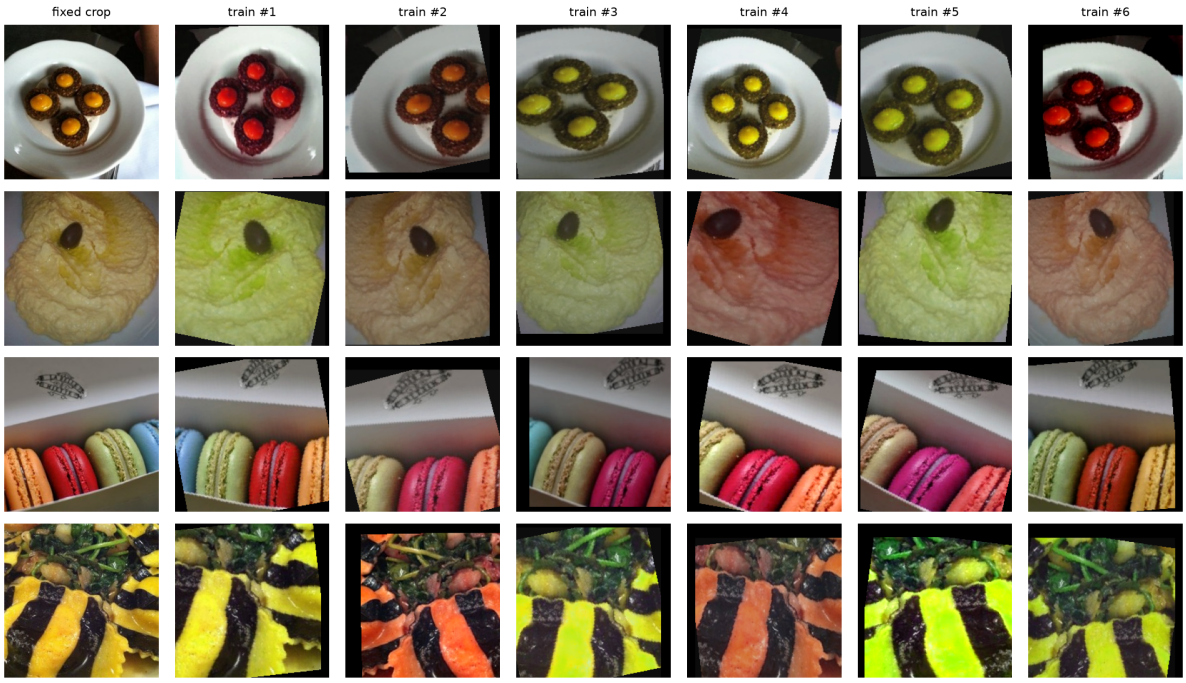
Chuẩn hoá. Ảnh được đưa về tensor trong $[0, 1]$ rồi chuẩn hoá với mean $(0.485, 0.456, 0.406)$ và std $(0.229, 0.224, 0.225)$ của ImageNet, đúng thống kê mà cả hai bộ trọng số pretrained được train. Chọn thống kê này cho mọi run, kể cả scratch, để đầu vào của phần mạng bị đóng băng ở chế độ head và partial có cùng thang đo như lúc pretrain; nếu dùng thang đo khác, BatchNorm đóng băng của ResNet-50 sẽ chuẩn hoá bằng thống kê không còn khớp, còn LayerNorm của DeiT-S thì tự điều chỉnh, làm lệch phép so sánh hai họ kiến trúc.

3.2 Tăng cường dữ liệu

Sau bước đó, chỉ tập train được tăng cường dữ liệu (Bảng 4, Hình 4); validation và test chỉ được chuẩn hoá.

Bước	Train	Validation, test và tập train dùng để đánh giá
Cắt cố định (một lần, mọi ảnh)	Đổi cạnh ngắn về 256 px, cắt giữa 224×224	như train
Cắt co giãn	Cắt ngẫu nhiên một vùng chiếm 50–100% diện tích ảnh 224×224 (tỉ lệ cạnh từ 3/4 đến 4/3), rồi đổi lại về 224×224	–
Lật	Lật ngang với xác suất 0.5	–
Xoay	Xoay ngẫu nhiên trong khoảng $\pm 15°$	–
Màu	Độ sáng, tương phản, độ bão hoà đổi trong $\pm 20\%$; sắc độ xoay trong ± 0.1 vòng màu ($\pm 36°$)	–
Dịch	Dịch ngẫu nhiên tối đa 10% theo mỗi chiều	–
Chuẩn hoá	Đưa về tensor trong $[0, 1]$, rồi trừ mean và chia std của ImageNet trên từng kênh	như train

Bảng 4: Pipeline dữ liệu dùng chung, theo đúng thứ tự áp dụng.



Hình 4: Minh hoạ tăng cường dữ liệu.

3.3 Cấu hình DataLoader

Mỗi run dùng bốn DataLoader của PyTorch (Bảng 5), cấu hình giống nhau ở cả 8 run. Tập train được xáo trộn lại ở mỗi epoch và dùng batch 128; các tập đánh giá giữ thứ tự cố định và dùng batch lớn hơn (256) để chạy nhanh. "Train (đánh giá)" là 20% tập train (135 ảnh mỗi lớp), chỉ dùng để đo accuracy trên train.

DataLoader	Số ảnh	Batch	Tăng cường
Train	68 175	128	có
Train (đánh giá)	13 635	256	không
Validation	7 575	256	không
Test	25 250	256	không

Bảng 5: Bốn DataLoader của mỗi run.

4 Cài đặt thí nghiệm

4.1 Các mô hình

Cả hai mô hình có cùng một khung: một **feature extractor** biến ảnh thành một vector đặc trưng mô tả nội dung ảnh, và một **classification head** biến vector đó thành 101 điểm số, mỗi điểm cho một món. Khác nhau ở feature extractor: ResNet-50 là mạng tích chập, quét ảnh bằng các filter nhỏ và thu nhỏ ảnh dần qua từng stage; DeiT-S là Transformer, cắt ảnh thành các patch rồi cho mọi patch trao đổi thông tin với nhau bằng self-attention. Mục 4.1.1 và 4.1.2 trình bày cấu trúc của từng mô hình và chức năng của các layer trong đó.

Hai mô hình dùng trọng số công bố trên Hugging Face (Bảng 6), có số tham số và chi phí tính toán gần nhau, và cùng được pretrain chỉ trên ImageNet-1k để nguồn pretrain không làm lệch so sánh. Classification head được thay bằng một linear layer 101 đầu ra, khởi tạo ngẫu nhiên. Số tham số và kích thước đầu ra dưới đây được đo trực tiếp trên mô hình.

Họ	Mô hình	Tham số (M)	FLOPs (G)	Nguồn pretrained
CNN	ResNet-50	23.7	8.2	`microsoft/resnet-50` (ImageNet-1k)
Transformer	DeiT-S	21.7	9.2	`facebook/deit-small-patch16-224` (ImageNet-1k)

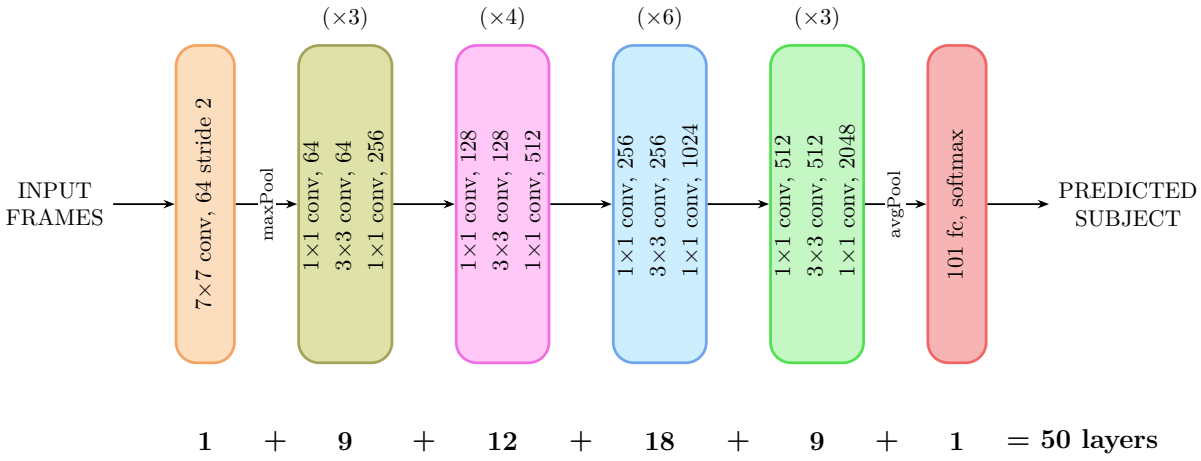
Bảng 6: Các mô hình sử dụng.

4.1.1 ResNet-50

ResNet-50 [3] gồm một *stem* (convolution 7×7 và max pooling, thu ảnh từ 224×224 xuống 56×56), bốn *stage* gồm 3, 4, 6 và 3 *bottleneck block*, rồi global average pooling và một fully connected layer 101 đầu ra (Hình 5). Từ stage 2, mỗi stage giảm kích thước ảnh một nửa (còn 28, 14, rồi 7) và tăng gấp đôi số kênh (từ 256 lên 2048).

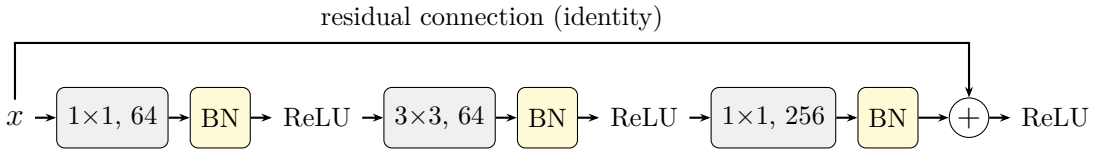
Các layer trong ResNet-50 và chức năng:

- **Convolution:** trượt các filter nhỏ trên ảnh để phát hiện đặc trưng cục bộ như cạnh, màu, kết cấu; convolution có stride 2 làm ảnh nhỏ đi một nửa.
- **Max pooling:** giữ giá trị lớn nhất trong mỗi vùng 3×3, dùng ở stem để thu nhỏ ảnh một nửa.
- **Batch normalization (BatchNorm):** đặt sau mỗi convolution, chuẩn hoá từng kênh theo trung bình và độ lệch chuẩn của cả batch, giúp train ổn định và nhanh hơn.
- **ReLU (activation function):** hàm kích hoạt phi tuyến, giữ số dương và đặt số âm về 0; thiếu nó, cả mạng chỉ là một phép biến đổi tuyến tính.
- **Residual connection** (*skip connection*): cộng đầu vào của block vào đầu ra, để block chỉ cần học phần thay đổi; nhờ vậy mạng 50 layer vẫn train được.
- **Global average pooling:** lấy trung bình mỗi kênh trên toàn ảnh, biến feature map $2048 \times 7 \times 7$ thành một vector 2048 số.
- **Fully connected layer và softmax:** fully connected layer biến vector 2048 số thành 101 điểm số (logit); softmax đổi chúng thành xác suất của 101 món.



Hình 5: Kiến trúc ResNet-50: mỗi hộp là một stage, ghi các convolution của một bottleneck block và số lần lặp ($\times n$); hàng dưới đếm số layer có trọng số của từng phần. Ở chế độ partial, stem, stage 1 và stage 2 (hộp thứ nhất đến thứ ba) bị đóng băng; classification head có 101 đầu ra.

Các layer này được ghép thành *bottleneck block* (Hình 6): convolution 1×1 đầu giảm số kênh (ví dụ 256 xuống 64) để convolution 3×3 giảm khối lượng tính toán, rồi convolution 1×1 cuối tăng lại gấp 4; ở block đầu mỗi stage, residual connection dùng thêm một convolution 1×1 (*projection shortcut*) để khớp kích thước.



Hình 6: Bottleneck block của ResNet-50 (ví dụ block thứ hai của stage 1, x có 256 kênh): ba convolution, mỗi convolution theo sau là BatchNorm (BN); đầu vào x được cộng vào đầu ra trước ReLU cuối. Ở block đầu của mỗi stage, nhánh residual connection là một convolution 1×1 kèm BatchNorm thay cho phép chép nguyên (identity).

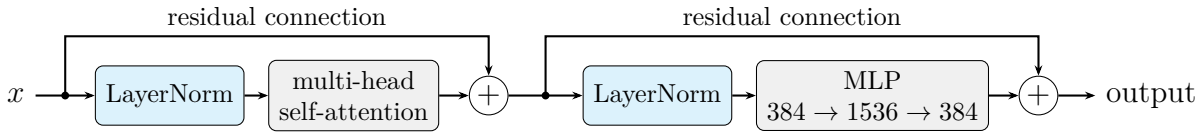
4.1.2 DeiT-S

DeiT-S (*Data-efficient image Transformer*, bản small) là một Vision Transformer [2] cỡ nhỏ [4]: 12 Transformer block, mỗi token 384 chiều, multi-head self-attention 6 head. Điểm mới của DeiT là cách huấn luyện giúp Transformer đạt accuracy tốt khi chỉ train trên ImageNet-1k. Ảnh 224×224 được chia thành 196 patch 16×16, mỗi patch được chiếu thành một token 384 chiều (*patch embedding*); thêm class token (CLS) và positional embedding, chuỗi 197 token đi qua 12 Transformer block với kích thước không đổi; cuối cùng class token qua LayerNorm và một fully connected layer cho ra 101 điểm số.

Các layer trong DeiT-S và chức năng của chúng:

- **Patch embedding:** một convolution 16×16 stride 16, tương đương một linear projection, biến mỗi patch thành một vector 384 chiều (một token).
- **Class token (CLS) và positional embedding:** class token là một vector học được, thêm vào đầu chuỗi để gom thông tin của cả ảnh cho bước phân loại; positional embedding cộng vào từng token để mô hình biết mỗi patch nằm ở đâu.
- **Layer normalization (LayerNorm):** chuẩn hoá từng token theo chính các giá trị của token đó, không phụ thuộc batch.
- **Multi-head self-attention:** gồm ba linear layer tạo query, key và value cho mỗi token, và một linear layer chiếu kết quả về 384 chiều. Nó cho mỗi token lấy thông tin từ mọi token khác, với trọng số tuỳ mức độ liên quan; nhờ vậy một patch nhìn được toàn ảnh ngay từ block đầu. 6 head chạy song song, mỗi head có thể chú ý đến một kiểu quan hệ khác nhau.
- **MLP:** hai linear layer (384 lên 1536 rồi về 384) với activation function GELU, một phiên bản mềm của ReLU, xử lý riêng từng token.
- **Residual connection:** như ở ResNet-50, cộng đầu vào của mỗi phần vào đầu ra của nó.
- **Fully connected layer và softmax:** biến class token thành 101 điểm số (logit) rồi thành xác suất.

Các layer này được ghép thành *Transformer block* (Hình 7): multi-head self-attention rồi MLP, mỗi phần có LayerNorm đứng trước và một residual connection.



Hình 7: Một Transformer block của DeiT-S (x là chuỗi 197 token × 384): LayerNorm đứng trước multi-head self-attention và MLP, và đầu vào của mỗi phần được cộng vào đầu ra của nó (residual connection).

4.2 Các chế độ đóng băng

Mỗi mô hình được huấn luyện theo bốn chế độ, khác nhau ở trọng số khởi tạo và phần mạng được cập nhật (Bảng 7). Hai mô hình nhân bốn chế độ thành 8 run, mỗi run train một lần với seed 42 trên toàn bộ tập train; dữ liệu, split, hàm loss và loại optimizer (AdamW) giống nhau ở cả bốn chế độ, còn learning rate, số epoch, warmup và dropout khác nhau theo chế độ (Mục 4.3).

Chế độ	Khởi tạo	Phần được học	Epoch dự tính	ResNet-50 trainable	DeiT-S trainable
scratch	ngẫu nhiên	toàn bộ mạng	100	23 714 981 (100%)	21 704 549 (100%)
head	pretrained	chỉ classification head	100	206 949 (0.9%)	38 885 (0.2%)
partial	pretrained	nửa sau của mạng	100	22 270 053 (93.9%)	10 686 437 (49.2%)
full	pretrained	toàn bộ mạng	100	23 714 981 (100%)	21 704 549 (100%)

Bảng 7: Bốn chế độ huấn luyện, số epoch dự tính và số tham số được học.

Scratch (train từ đầu). Mô hình khởi tạo ngẫu nhiên, không dùng trọng số pretrained, và mọi tham số đều được học. Đây là mốc cho biết mô hình học được gì chỉ từ khoảng 68 000 ảnh Food-101, và dùng để đo pretrain giúp được bao nhiêu. Mọi chế độ dùng cùng số epoch tối đa (100) và early stopping quyết định thời điểm dừng; scratch thường chạy lâu nhất vì phải học mọi thứ từ đầu.

Head (chỉ train classification head). Mô hình nạp trọng số pretrained ImageNet-1k, classification head được thay bằng một linear layer 101 đầu ra khởi tạo ngẫu nhiên, và *chỉ layer này được học*; mọi phần còn lại bị đóng băng. Mô hình dùng nguyên các đặc trưng đã học từ ImageNet và chỉ học cách đọc chúng cho 101 món, nên chỉ có 0.9% (ResNet-50) và 0.2% (DeiT-S) số tham số được cập nhật. Đây là chế độ rẻ nhất và cho biết các đặc trưng pretrained đã tốt đến đâu với Food-101.

Partial (đóng băng nửa đầu). Mô hình nạp trọng số pretrained, nửa đầu của mạng (tính theo độ sâu) bị đóng băng, còn nửa sau và classification head được học. Nửa đầu giữ các đặc trưng cấp thấp như cạnh và kết cấu; nửa sau được điều chỉnh cho món ăn. Với ResNet-50, phần đóng băng gồm stem, stage 1 và stage 2; với DeiT-S, gồm patch embedding, class token, positional embedding và 6 Transformer block đầu. Vì ResNet-50 dồn tham số ở hai stage cuối còn 12 block của DeiT-S có cùng kích thước, cùng là nửa đầu nhưng ResNet-50 vẫn học 93.9% tham số còn DeiT-S học 49.2%.

Full (finetune toàn bộ). Mô hình nạp trọng số pretrained rồi cập nhật *mọi* tham số với learning rate nhỏ, nên các đặc trưng pretrained được điều chỉnh dần cho Food-101. Đây là cách dùng thông dụng nhất của pretrain và thường cho accuracy cao, với chi phí

tính toán bằng chế độ scratch trên mỗi epoch nhưng thường hội tụ sau ít epoch hơn scratch.

4.3 Siêu tham số huấn luyện

Cả 8 run dùng chung một công thức; chỉ learning rate, warmup và dropout đổi theo chế độ. Mọi giá trị nằm trong Bảng 8 và giống nhau ở ResNet-50 và DeiT-S.

Tham số	scratch	head	partial	full
Optimizer	AdamW, weight decay 0.05			
Hàm loss	cross-entropy, label smoothing 0.2			
Lịch learning rate	warmup tuyến tính rồi cosine về 0, cập nhật mỗi iteration			
Batch (train / đánh giá)	128 / 256			
Độ chính xác số, seed	bf16 autocast, seed 42			
Epoch tối đa	100			
Early stopping (patience)	10 epoch theo validation accuracy			
Learning rate	5×10^{-4}	1×10^{-3}	2×10^{-4}	1×10^{-4}
Warmup (epoch)	5	1	1	1
Dropout (trước lớp phân loại)	0.4	0.2	0.3	0.4

Bảng 8: Siêu tham số. Ba dòng cuối khác nhau theo chế độ. Patience là số epoch liên tiếp không có validation accuracy tốt hơn mà run còn chịu được; checkpoint giữ lại luôn là epoch có validation accuracy cao nhất, nên 100 là số epoch tối đa. Dropout đặt trên đặc trưng đã gộp ngay trước lớp phân loại cuối, cùng một vị trí ở hai kiến trúc. Weight decay không áp dụng cho bias, scale của lớp chuẩn hoá, class token và positional embedding.

Lý do các tham số khác nhau giữa các chế độ:

- **Learning rate** giảm dần theo số trọng số pretrained được cập nhật: head (10^{-3}) chỉ học một lớp tuyến tính khởi tạo ngẫu nhiên nên chịu được bước lớn; partial và full (2×10^{-4} và 10^{-4}) cập nhật cả các trọng số đã học tốt từ ImageNet nên cần bước nhỏ để không phá chúng. Scratch (5×10^{-4}) khởi tạo ngẫu nhiên toàn bộ nên dùng giá trị lớn hơn.
- **Warmup** dài hơn ở scratch (5 epoch so với 1): khi mọi trọng số đều ngẫu nhiên, vài bước đầu với learning rate lớn dễ làm huấn luyện bất ổn, nhất là với Transformer.
- **Dropout** tăng theo số trọng số được học: 0.2 ở head, 0.3 ở partial, 0.4 ở full và scratch.

4.4 Độ đo đánh giá

Cả 8 run được đánh giá theo cùng một quy trình:

- **Chọn checkpoint:** giữ epoch có validation accuracy cao nhất, rồi chấm checkpoint đó một lần trên test. Test không dùng cho bất kỳ quyết định nào.
- **Chất lượng:** test accuracy là chỉ số chính; kèm macro precision, recall, F1 và accuracy từng lớp.
- **Chi phí:** số tham số được học, thời gian mỗi epoch, bộ nhớ GPU đỉnh và tốc độ suy luận.

- **Phân tích lỗi:** confusion matrix, các cặp lớp bị nhầm nhiều nhất và các ảnh sai với độ tự tin cao, trên mô hình CNN và Transformer tốt nhất theo validation.
- **Các so sánh:** DeiT-S với ResNet-50 trong từng chế độ; full với scratch (pretrain giúp bao nhiêu); head và partial với full (mất bao nhiêu khi đóng băng); chi phí đặt cạnh accuracy.

5 Kết quả và phân tích

Mục này đọc tám run (2 mô hình với 4 chế độ mỗi mô hình). Mỗi cấu hình được train một lần với seed 42. Checkpoint đem ra test là epoch có validation accuracy cao nhất, và test không tham gia chọn epoch, mô hình hay siêu tham số. Chỉ số chính là test accuracy trên 25 250 ảnh.

Trong cùng một chế độ, ResNet-50 và DeiT-S dùng chung tập ảnh, pipeline, hàm loss, AdamW, learning rate, warmup và dropout. Validation và test luôn cố định, nên mọi run được chấm trên đúng cùng các ảnh.

5.1 Độ chính xác

Bảng 9 gồm accuracy, F1 và chi phí của 8 run; Bảng 10 gồm precision, recall và F1 macro; Bảng 11 cho thấy test accuracy thay đổi bao nhiêu khi đổi chế độ.

Mô hình	Chế độ	Tham số học	Val acc (%)	Test acc (%)	Test F1 (%)	s/epoch	GB	img/s
DeiT-S	scratch	21.70M	43.6	47.7	47.5	80	4.9	2856
DeiT-S	head	0.04M	62.5	66.3	65.9	48	**0.5**	**2864**
DeiT-S	partial	10.69M	**79.5**	**83.6**	**83.6**	56	2.6	2858
DeiT-S	full	21.70M	79.4	83.5	**83.6**	84	4.9	2859
ResNet-50	scratch	23.71M	70.7	75.5	75.4	99	5.7	2282
ResNet-50	head	0.21M	62.7	66.7	66.4	**46**	0.9	2279
ResNet-50	partial	22.27M	77.1	81.7	81.7	55	1.9	2279
ResNet-50	full	23.71M	77.5	81.8	81.8	98	5.7	2281

Bảng 9: Kết quả của 8 run: số tham số được học, validation và test accuracy, F1 macro, thời gian mỗi epoch, bộ nhớ GPU đỉnh và tốc độ suy luận trên GPU A10G. In đậm: giá trị tốt nhất của mỗi cột (accuracy, F1 và img/s cao nhất; s/epoch và GB thấp nhất).

Mô hình	Chế độ	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
DeiT-S	scratch	47.7	49.1	47.7	47.5
DeiT-S	head	66.3	67.9	66.3	65.9
DeiT-S	partial	**83.6**	**84.3**	**83.6**	**83.6**
DeiT-S	full	83.5	84.0	83.5	**83.6**
ResNet-50	scratch	75.5	75.7	75.5	75.4
ResNet-50	head	66.7	67.5	66.7	66.4
ResNet-50	partial	81.7	81.9	81.7	81.7
ResNet-50	full	81.8	82.0	81.8	81.8

Bảng 10: Accuracy, precision, recall và F1 macro (trung bình trên 101 lớp) trên test. In đậm: giá trị cao nhất của mỗi cột.

		ResNet-50			DeiT-S		
Nội dung so sánh	Chuyển chế độ	Từ	Đến	Thay đổi	Từ	Đến	Thay đổi
Pretrain giúp được bao nhiêu	scratch → full	75.5	81.8	**+6.3**	47.7	83.5	**+35.8**
Chỉ train head mất bao nhiêu	full → head	81.8	66.7	**−15.1**	83.5	66.3	**−17.2**
Đóng băng nửa đầu mất bao nhiêu	full → partial	81.8	81.7	**−0.1**	83.5	83.6	**+0.1**

Bảng 11: Test accuracy (%) khi chuyển từ chế độ này sang chế độ khác. "Từ" và "Đến" lấy từ Bảng 9; "Thay đổi" là hiệu của hai số đó, tính bằng điểm phần trăm.

Trên test, DeiT-S partial đạt 83.6% và DeiT-S full đạt 83.5%. Hai run ResNet-50 tương ứng đạt 81.7% và 81.8%. Thấp nhất là DeiT-S scratch, 47.7%. Macro precision, recall và F1 của mỗi run đều sát test accuracy của run đó (ví dụ DeiT-S head: accuracy 66.3%, precision 67.9%, F1 65.9%) và giữ nguyên thứ tự này, nên accuracy mô tả được chất lượng trên cả 101 lớp. Test accuracy luôn cao hơn validation accuracy ở cả tám run (ví dụ DeiT-S partial: 79.5% trên validation, 83.6% trên test); một lý do có thể là validation lấy từ train còn nhiễu hơn test (Mục 2.4).

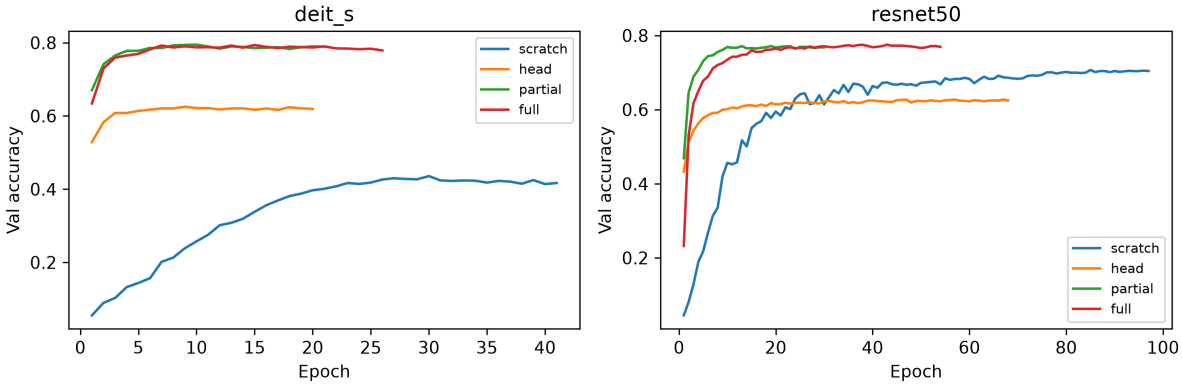
Hai họ trong cùng một chế độ. Khi finetune, DeiT-S đạt 83.5% ở full và 83.6% ở partial, cao hơn ResNet-50 (81.8% và 81.7%). Ở chế độ head, DeiT-S đạt 66.3% và ResNet-50 đạt 66.7%, dù DeiT-S chỉ học 38 885 tham số còn ResNet-50 học 206 949. Khi train từ đầu, ResNet-50 đạt 75.5% còn DeiT-S chỉ đạt 47.7%; đây là khoảng cách lớn nhất giữa hai họ.

Pretrain. So với scratch, full tăng test accuracy từ 47.7% lên 83.5% ở DeiT-S và từ 75.5% lên 81.8% ở ResNet-50. DeiT-S phụ thuộc vào trọng số pretrained nhiều hơn ResNet-50. Một cách giải thích thường gặp: phép tích chập vốn chỉ nhìn vùng ảnh lân cận và dùng cùng một filter ở mọi vị trí, nên ResNet-50 đã có sẵn hai giả định hợp với ảnh (đặc trưng nằm ở vùng gần nhau, và một vật ở đâu trong ảnh cũng nhận ra như nhau). DeiT-S không có hai giả định này mà phải học từ dữ liệu, nên cần nhiều ảnh hơn; 68 175 ảnh là ít đối với một Transformer.

Đóng băng so với full. Chỉ train head làm test accuracy giảm từ 81.8% xuống 66.7% ở ResNet-50 và từ 83.5% xuống 66.3% ở DeiT-S. Giữ nguyên đặc trưng ImageNet và chỉ

học lớp phân loại chưa đủ cho 101 món. Partial và full gần như ngang nhau ở cả hai mô hình (81.7% so với 81.8% ở ResNet-50, 83.6% so với 83.5% ở DeiT-S).

5.2 Quá trình huấn luyện và chi phí



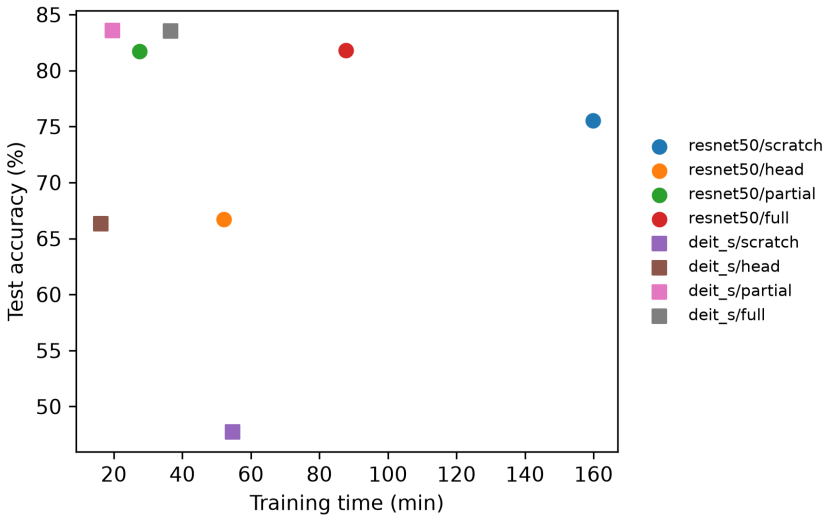
Hình 8: Validation accuracy theo epoch của 8 run. Mỗi đường kết thúc ở epoch run dừng do early stopping.

Mô hình	Chế độ	Epoch tốt nhất	Epoch dừng	Train (%)	Val (%)
ResNet-50	scratch	85	97	99.8	70.7
ResNet-50	head	57	68	66.3	62.7
ResNet-50	partial	19	30	92.3	77.1
ResNet-50	full	43	54	94.7	77.5
DeiT-S	scratch	30	41	67.4	43.6
DeiT-S	head	9	20	63.2	62.5
DeiT-S	partial	10	21	93.3	79.5
DeiT-S	full	15	26	95.6	79.4

Bảng 12: Epoch có validation accuracy cao nhất, epoch run dừng (epoch tốt nhất cộng 11, vì run dừng khi số epoch liên tiếp không cải thiện vượt patience 10; riêng ResNet-50 scratch cộng 12 vì được chạy làm hai đợt), và accuracy trên train và validation (Val) ở epoch tốt nhất.

Hình 8 và Bảng 12 cho thấy quá trình huấn luyện của 8 run. DeiT-S scratch đạt validation cao nhất ở epoch 30, rồi đi ngang quanh 41–43% và dừng ở epoch 41. Train accuracy tại epoch đó chỉ 67.4%. ResNet-50 scratch tiếp tục tăng validation đến epoch 85. Tại đó train accuracy là 99.8% còn validation là 70.7%.

Ở chế độ head, train và validation đều nằm trong khoảng 62–67% (DeiT-S: 63.2% và 62.5%; ResNet-50: 66.3% và 62.7%). Một lớp tuyến tính không đủ sức đưa accuracy lên mức của partial và full. Partial và full khớp tập train ở mức 92–96%, với validation 77–80%. Partial khớp tập train ít hơn full (ResNet-50 từ 94.7% xuống 92.3%, DeiT-S từ 95.6% xuống 93.3%) trong khi validation gần như giữ nguyên; một khả năng là phần đầu mạng không được cập nhật nên mô hình bớt khớp tập train.



Hình 9: Test accuracy theo tổng thời gian các bước cập nhật (phút, GPU A10G, bf16). Thời gian này không gồm lượt chấm validation.

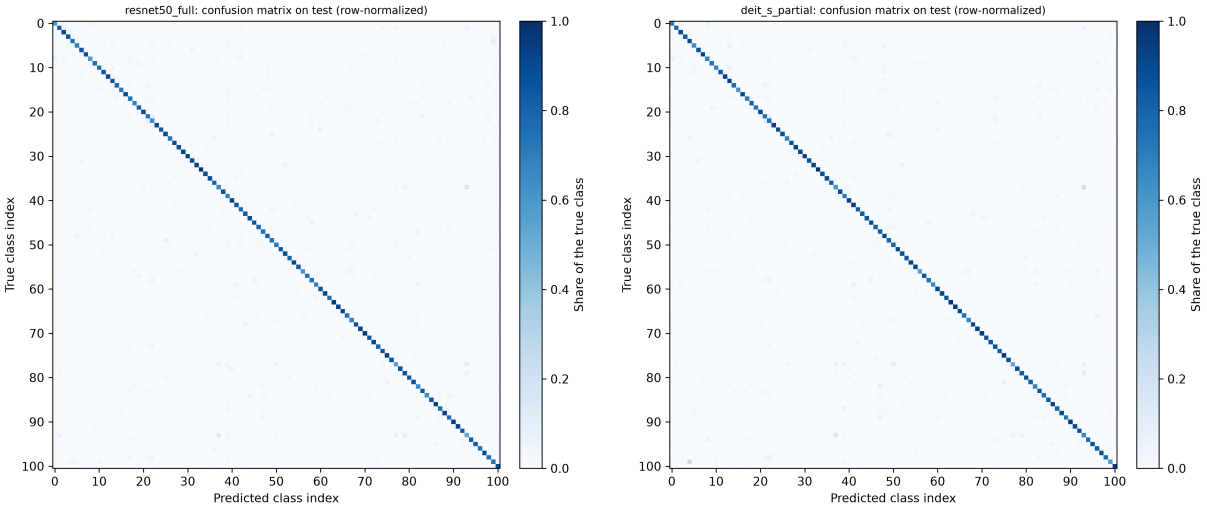
Chi phí nằm ở các cột cuối của Bảng 9 và ở Hình 9. So với full, partial rẻ hơn mà accuracy không giảm:

- **Thời gian mỗi epoch:** ResNet-50 từ 98 s xuống 55 s; DeiT-S từ 84 s xuống 56 s.
- **Bộ nhớ GPU đỉnh:** ResNet-50 từ 5.7 GB xuống 1.9 GB; DeiT-S từ 4.9 GB xuống 2.6 GB.
- **Tổng thời gian train đến khi dừng:** ResNet-50 từ 88 phút xuống 28 phút; DeiT-S từ 37 phút xuống 20 phút.

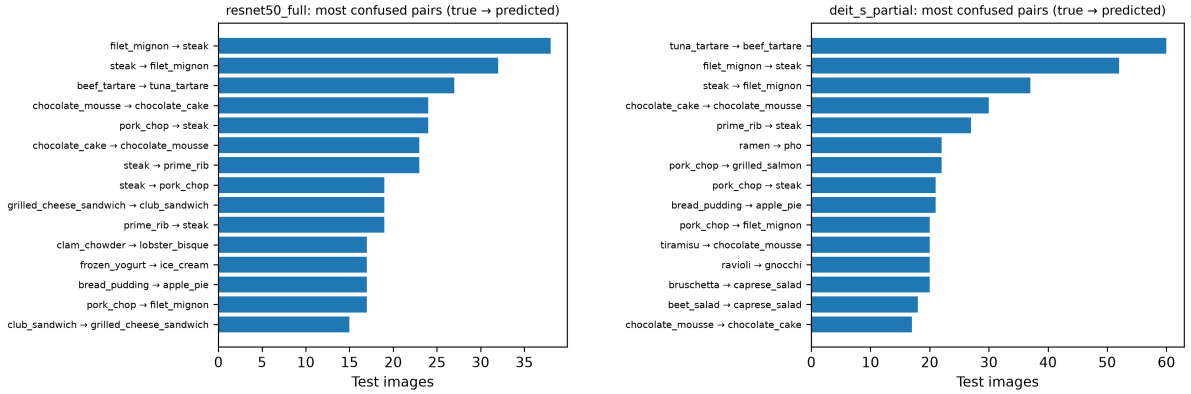
Tốn nhất là train từ đầu, ví dụ ResNet-50 scratch mất 160 phút. Khi suy luận, DeiT-S xử lý khoảng 2 860 ảnh/giây, ResNet-50 khoảng 2 280 ảnh/giây, tức DeiT-S nhanh hơn khoảng 25%.

5.3 Phân tích lỗi

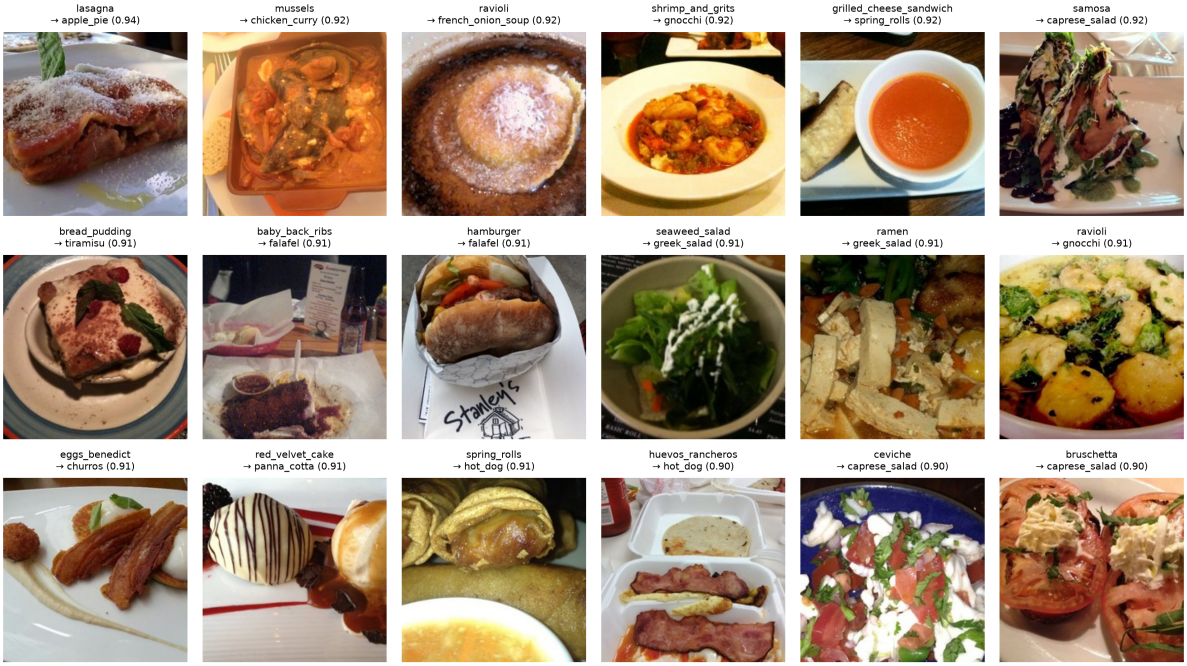
Phần này dùng mô hình validation cao nhất của mỗi họ, ResNet-50 full và DeiT-S partial.



Hình 10: Ma trận nhầm lẫn trên test (hàng là nhãn thật, cột là nhãn đoán). Trái: ResNet-50 full. Phải: DeiT-S partial.



Hình 11: 15 cặp lớp bị nhầm nhiều nhất (nhãn thật → nhãn đoán) trên test. Trái: ResNet-50 full. Phải: DeiT-S partial.



Hình 12: 18 ảnh test bị DeiT-S partial đoán sai với độ tự tin cao nhất (nhãn thật → nhãn đoán, xác suất trong ngoặc).

Ma trận đầy đủ ở Hình 10. Hai mô hình nhầm các món trông gần nhau (Hình 11): `filet_mignon` với `steak`, `chocolate_cake` với `chocolate_mousse`, `pork_chop` với `steak`, `beef_tartare` với `tuna_tartare`. DeiT-S partial đoán `tuna_tartare` thành `beef_tartare` ở 60 trên 250 ảnh của lớp. Lớp dễ nhất với cả hai mô hình là `edamame` (99.6% ở ResNet-50 full, 100% ở DeiT-S partial); tiếp theo là `pad_thai` và `macarons` ở ResNet-50, `macarons` và `pho` ở DeiT-S. Khó nhất với ResNet-50 full là `steak`, `pork_chop` và `bread_pudding` (51.6–58.0%); với DeiT-S partial là `pork_chop`, `huevos_rancheros` và `tuna_tartare` (54.0–57.2%). Trong 18 ảnh sai với xác suất cao nhất (Hình 12) có cả ảnh nhiều món hoặc nhãn khó đọc, và cả lỗi thật như `hamburger` bị đoán thành `falafel`.

Mode	Cả hai đúng	Chỉ ResNet	Chỉ DeiT	Cả hai sai	Jaccard lỗi	Sai cùng lớp
scratch	43.6	32.0	4.2	20.3	0.36	16.8
head	56.1	10.6	10.2	23.1	0.53	38.3
partial	75.4	6.4	8.2	10.1	0.41	41.7
full	75.5	6.3	8.1	10.1	0.41	41.8

Bảng 13: ResNet-50 so với DeiT-S trên cùng 25 250 ảnh test (%, trừ Jaccard). Jaccard lỗi: số ảnh cả hai cùng sai chia cho số ảnh có ít nhất một mô hình sai.

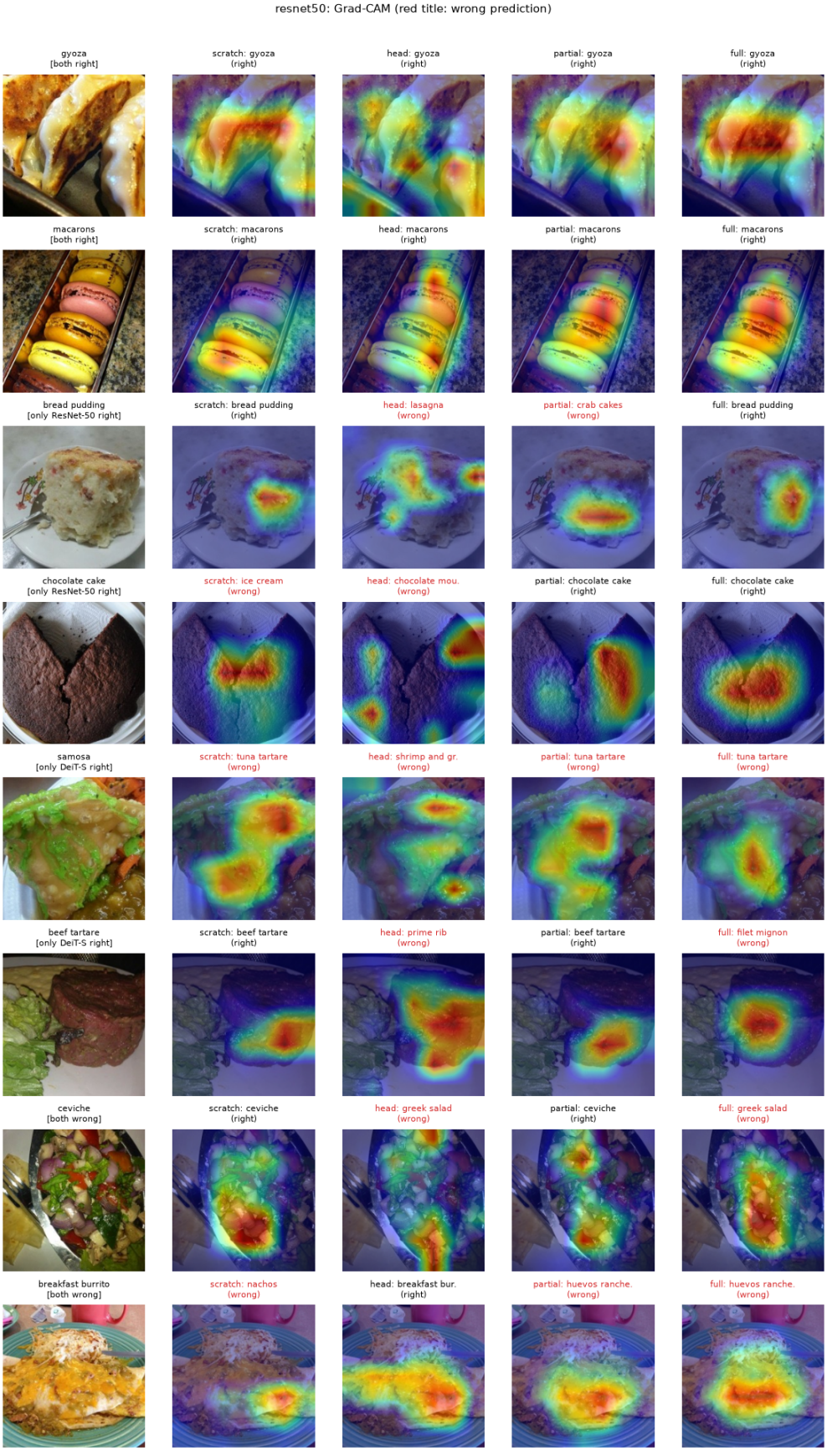
Ở partial và full, khoảng 10% ảnh bị cả hai cùng sai, 6.3–6.4% chỉ ResNet-50 đúng và 8.1–8.2% chỉ DeiT-S đúng (Bảng 13); Jaccard lỗi là 0.41.

5.4 Giải thích dự đoán của mô hình

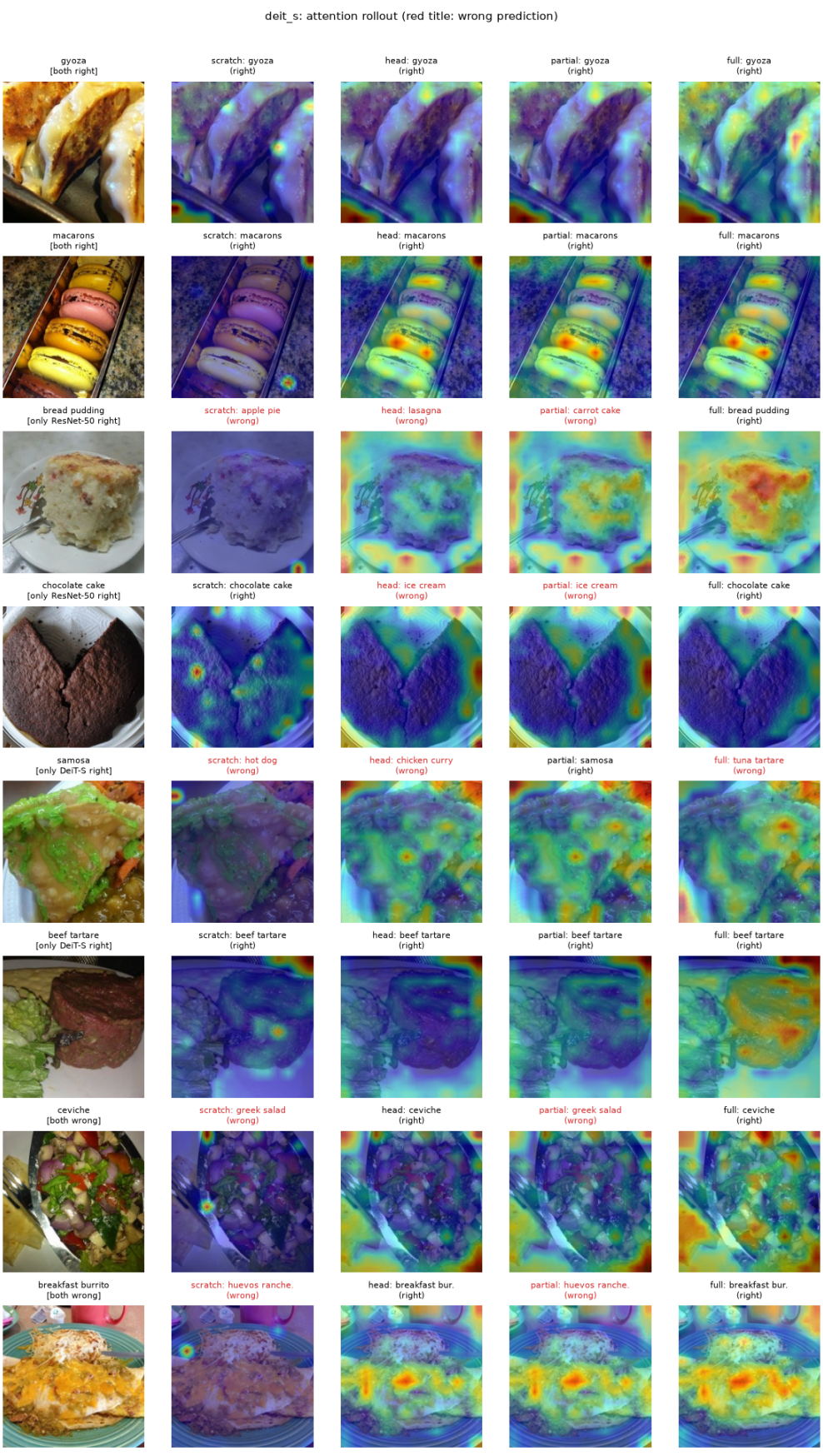
ResNet-50 dùng Grad-CAM trên stage cuối. DeiT-S dùng attention rollout, không phụ thuộc lớp được đoán. Hai hình dưới dùng chung 8 ảnh test.

Model	Mode	Gốc (%)	Xóa 25% nổi bật	Xóa 25% ngẫu nhiên	Xóa 50% nổi bật	Xóa 50% ngẫu nhiên
resnet50	scratch	74.1	25.7	18.6	7.4	3.6
resnet50	head	67.0	48.9	61.1	29.5	50.4
resnet50	partial	80.5	47.1	72.4	13.4	50.1
resnet50	full	81.3	40.0	70.8	11.7	35.4
deit_s	scratch	47.5	28.7	37.8	12.7	24.8
deit_s	head	65.8	61.9	61.2	52.8	53.1
deit_s	partial	83.4	78.0	80.7	64.0	72.4
deit_s	full	83.2	71.2	77.6	50.3	66.1

Bảng 14: Phép xoá trên 2 000 ảnh test: accuracy (%) khi giữ nguyên ảnh, và khi xoá 25% hoặc 50% số ô theo thứ tự của bản đồ giải thích (nổi bật) hoặc ngẫu nhiên.



Hình 13: Grad-CAM của bốn chế độ ResNet-50 trên 8 ảnh test.



Hình 14: Attention rollout của bốn chế độ DeiT-S trên cùng 8 ảnh.

Ở partial và full, vùng nóng của cả hai mô hình nằm trên món (Hình 13 và 14). ResNet-50 head loang hơn, có ảnh rơi vào đĩa hoặc góc ảnh. DeiT-S scratch chỉ còn vài đốm ở rìa. Phép xoá trên 2 000 ảnh kiểm tra bản đồ này (Bảng 14): xoá 25% ô nổi bật làm ResNet-50 full còn 40.0%, xoá ngẫu nhiên còn 70.8%. DeiT-S full còn 71.2% và 77.6%. Ở head, đường xoá theo rollout của DeiT-S gần như trùng đường xoá ngẫu nhiên. Xoá ngẫu nhiên 50% ô, DeiT-S partial và full còn 72.4% và 66.1%, ResNet-50 partial và full còn 50.1% và 35.4%.

6 Thảo luận, kết luận và hạn chế

6.1 Kết luận chính

- **DeiT-S tốt hơn ResNet-50 khi finetune từ ImageNet, còn ResNet-50 tốt hơn khi train từ đầu.** Khi finetune, DeiT-S đạt 83.5–83.6% test accuracy, ResNet-50 đạt 81.7–81.8% (Mục 5.1); DeiT-S suy luận nhanh hơn khoảng 25% (Mục 5.2). Khi train từ đầu, ResNet-50 đạt 75.5%, DeiT-S chỉ đạt 47.7%.
- **Transformer cần pretrain trên tập cỡ này; CNN cũng được lợi nhưng ít hơn.** Với 68 175 ảnh train, full tăng test accuracy từ 47.7% lên 83.5% ở DeiT-S và từ 75.5% lên 81.8% ở ResNet-50.
- **Đóng băng nửa đầu mạng cho accuracy ngang finetune toàn bộ mà rẻ hơn; chỉ train head chỉ hợp khi thiếu tài nguyên.** Partial giảm 33–44% thời gian mỗi epoch và 47–67% bộ nhớ đỉnh so với full (Mục 5.2). Chỉ train head làm test accuracy giảm từ 81.8% xuống 66.7% ở ResNet-50 và từ 83.5% xuống 66.3% ở DeiT-S.
- **Hai mô hình chủ yếu nhầm các món trông giống nhau, và sai ở những ảnh khá khác nhau.** Ví dụ `steak` với `filet_mignon`; Jaccard lỗi giữa hai mô hình là 0.41 (Mục 5.3).

6.2 Hạn chế

- **Mỗi cấu hình chỉ train một lần.** Kết quả ứng với một seed duy nhất, nên các lần chạy khác có thể cho số hơi lệch.
- **Công thức huấn luyện chưa được tối ưu cho từng mô hình.** Hai họ dùng chung một công thức và siêu tham số được dò rất ít, nên kết quả, nhất là ở chế độ scratch, phản ánh cả công thức chứ không chỉ kiến trúc.
- **Mỗi họ chỉ có một mô hình.** Kết luận về CNN và Transformer chỉ chắc cho ResNet-50 và DeiT-S, chưa chắc cho các kích thước hay kiến trúc khác.
- **Chỉ một tập dữ liệu và một nguồn pretrain.** Food-101 có các lớp gần với ImageNet (Mục 2.1), nên lợi ích của pretrain có thể khác trên miền dữ liệu khác.
- **Phần giải thích ở quy mô nhỏ.** Grad-CAM, attention rollout và phép xoá ô chỉ dùng một tập con ảnh test và là phương pháp xấp xỉ, nên mang tính tham khảo.

- **Chi phí chỉ đo trên một cấu hình phần cứng.** Thời gian và bộ nhớ đo trên một GPU A10G với bf16, nên có thể khác ở phần cứng khác.

6.3 Khả năng tái lập

- **Cấu hình tập trung.** Cấu hình của 16 run (8 train, 8 test) được sinh từ một nơi duy nhất, `scripts/a1/make_configs.py`, với seed 42 và split cố định lưu thành tệp CSV.
- **Huấn luyện và đánh giá bằng script.** `scripts/train.py` huấn luyện theo từng tệp cấu hình; các script trong `scripts/a1/` chấm checkpoint trên test, sinh bảng, hình và các phân tích bổ sung. Mọi lựa chọn ngẫu nhiên đều có seed cố định.
- **Môi trường.** Các run chạy trên GPU A10G; chạy lại có thể lệch nhẹ ở chữ số thập phân của accuracy do tính không tất định của phép toán GPU và bf16.
- **Cách chạy lại.** Hướng dẫn từng bước nằm trong mục *Chạy lại A1* của `README.md`. Checkpoint của A1 không được công bố, nên muốn có kết quả thì phải huấn luyện lại.

Tài liệu

[1] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – mining discriminative components with random forests. In *European Conference on Computer Vision (ECCV)*, pages 446–461, 2014.

[2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[4] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning (ICML)*, pages 10347–10357, 2021.